

## Day 32



**Javascript**  
THE LANGUAGE OF THE WEB

### Async/Await in JavaScript:

#### What is async/await?

Async/await is a cleaner, easier way to work with Promises. It helps you write asynchronous code that looks like normal synchronous code.

#### What is async?

A function becomes “async” when you write:

```
async function myFunc() {}
```

Inside an async function, you can use await. An async function always returns a Promise.

#### What is await?

await pauses the function and waits for a Promise to finish.

```
let result = await promise;
```

Your code stops at this line until the promise resolves.

Example: Even though you just returned a string, the function returns a Promise automatically.

```
JS main.js > ...
1  async function greet() {
2    return "Hello";
3  }
4
5  greet().then(console.log); // prints: Hello
```

PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

```
● PS E:\JavaScript\Day31-40\Day32> node main.js
Hello
```

In this code, the function greet is marked as async, which means it automatically returns a Promise even though it simply returns the string "Hello". When greet() is called, it doesn't return the string directly—it returns a Promise that eventually resolves to "Hello". Since it's a Promise, we use .then() to get the final value. So greet().then(console.log) waits for the Promise to resolve and then prints "Hello" to the console.

Example: introducing the “await” in the code.

```
8  async function demo() {
9      let promise = new Promise((resolve) => {
10         resolve("Hello World");
11     });
12
13     let result = await promise; // waits for the promise
14     console.log(result);
15 }
16
17 demo();
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

● PS E:\JavaScript\Day31-40\Day32> node main.js  
Hello World

Here we create an async function called demo, which allows us to use the await keyword inside it. Inside the function, a promise is created that immediately resolves with the text "Hello World". When the code reaches await promise, the function pauses and waits for that promise to finish. Once the promise resolves, the result is stored in the variable result, and finally the program prints "Hello World" to the console.

## **Error Handling-try and catch:**

### **What is try...catch?**

try and catch are used to handle errors in JavaScript so that your program does not crash when something goes wrong.

### **How it works**

- The code inside try runs normally.
- If an error happens, the program immediately jumps to catch.
- The code inside catch handles the error safely.

### **Why is try...catch useful?**

- Prevents your program from breaking
- Helps you show user-friendly messages
- Lets you handle unexpected errors safely

Example: basic example where there is no error!

```
JS error.js > ...
1  setTimeout(() => {
2    |   console.log("This is a delayed message 1.");
3    | }, 1000);
4
5  setTimeout(() => {
6    |   console.log("This is a delayed message 2.");
7    | }, 2000);
8
9  setTimeout(() => {
10   |   console.log("This is a delayed message 3.");
11   | }, 3000);

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS E:\JavaScript\Day31-40\Day32> node .\error.js
This is a delayed message 1.
This is a delayed message 2.
This is a delayed message 3.
```

Example: basic example where there is error but no error handling.

```
JS error.js > ...
1  setTimeout(() => {
2    |   console.log("This is a delayed message 1.");
3    | }, 1000);
4
5  console.log(Aditya); //will generate ReferenceError: Aditya is not defined
6
7  setTimeout(() => {
8    |   console.log("This is a delayed message 2.");
9    | }, 2000);
10
11 setTimeout(() => {
12   |   console.log("This is a delayed message 3.");
13   | }, 3000);

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS E:\JavaScript\Day31-40\Day32> node .\error.js
console.log(Aditya);
^
ReferenceError: Aditya is not defined
    at Object.<anonymous> (E:\JavaScript\Day31-40\Day32\error.js:5:13)
    at Module.compile (node:internal/modules/cjs/loader:1730:14)
```

Example: now handling error using the try-catch

```
JS error.js > ...
1  setTimeout(() => {
2    |   console.log("This is a delayed message 1.");
3    | }, 1000);
4
5  try {
6    |   console.log(Aditya); //will generate ReferenceError: Aditya is not defined
7    | } catch (error) {
8    |   console.log("An error occurred:", error.message);
9    | }
10
11 setTimeout(() => {
12   |   console.log("This is a delayed message 2.");
13   | }, 2000);
14
15 setTimeout(() => {
16   |   console.log("This is a delayed message 3.");
17   | }, 3000);

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS E:\JavaScript\Day31-40\Day32> node .\error.js
An error occurred: Aditya is not defined
This is a delayed message 1.
This is a delayed message 2.
This is a delayed message 3.
```

Also, try...catch works only for synchronous code. But setTimeout runs after the try block is already done, so try...catch cannot catch its errors.

Example: setTimeout inside the try...catch.

```
18
19 try {
20     setTimeout(() => {
21         console.log(x); // x is not defined
22     }, 1000);
23 } catch (error) {
24     console.log("Caught error:", error);
25 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS E:\JavaScript\Day31-40\Day32> node .\error.js
● E:\JavaScript\Day31-40\Day32\error.js:21
    console.log(x); // x is not defined
      ^
ReferenceError: x is not defined
    at Timeout._onTimeout (E:\JavaScript\Day31-40\Day32\error.js:21:21)
    at listOnTimeout (node:internal/timers:588:17)
    at process.processTimers (node:internal/timers:523:7)
```

But, how can we rectify this? We can do so by writing another try...catch inside the setTimeout():

```
28 try {
29     setTimeout(() => {
30         try {
31             console.log(x); // x is not defined → error
32         } catch (err) {
33             console.log("Caught inside setTimeout:", err.message);
34         }
35     }, 1000);
36 } catch (error) {
37     console.log("Caught outside:", error.message);
38 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
● PS E:\JavaScript\Day31-40\Day32> node .\error.js
Caught inside setTimeout: x is not defined
```

--The End--